

Python Bytes Activity Packet

This little booklet will give you some Python to think about and code to write in between club meetings. If you would like more challenges: Talk to the coaches!

Claim A Prize

Python Bytes introduces a new feature that encourages you to code at home: Tier 1 prizes. To claim your Tier 1 prize simply code for at least an hour on your own (not during club meetings!)

Then come to the club meeting on a Wednesday night and tell us how it went. That's it! Now you can claim a Tier 1 prize from among the prize assortment.



But wait! If there is a Tier 1 then there must also be a Tier 2, right? Tier 2 prizes are one tier more luxurious. To claim a Tier 2 there are three steps. First you must agree with one of the coaches on a good project for you. This requires some planning ahead on your part and an all-important coach chat. The second step: Put effort into the project in your spare time to get it working. The third step is: Bring your resulting code to

the club and show the coach how it works. That's it. Offer good while Tier 2 prizes last! And between you and me: The real reward is the project.

Gargutron

Introducing a game we placed on the internet specifically for Python Bytes students to play: **Gargutron!** To play: open a browser and type in the website

<https://pythonbytes.azurewebsites.net/api/Gargutron>

Then follow directions. Hint: The first step is to add some more text to the link above. Gargutron will tell you how. Now on to Python: Playing Gargutron does not count as Tier 1 Prize effort because playing Gargutron is its own reward. However you can definitely write Python code that plays the Gargutron game and in that way earn a Tier 1 prize. Here is how you go about it.

Gargutron code step 1: Check `requests`

Write a one-line program and run it to make sure you have the Python `requests` library available. The one line of code is simply:

```
import requests
```

If this runs without complaining: Continue. If it gives you an error: You must install the `requests` library.

Gargutron code step 2: Set up an `input()` to format a guess

As you know you can input a string from a human being using a line of code like this:

```
guess = input('Input a guess: ')
```

Now you want to format a URL by adding two strings together. The first string is all the stuff leading up to the guess... and the second string is the guess. So this will look like this:

```
url = "https://pythonbytes.azurewebsites.net/api/Gargutron?food=" + guess
```

Now you can pass this url (a string) to the `requests.get()` method:

```
reply = requests.get(url)
```

This variable `reply` is called a *Response object*. This Response object contains information about what Gargutron had to say in reply to your message. You have used the `requests` library to `get` a Response from Gargutron. Now we unpack this Response.

Gargutron code step 3: Print Gargutron's reply

Gargutron's reply should be stored in `reply` (from the previous step) as text; so we can look at that by using a print statement like this:

```
print(reply.text)
```

The `.text` is the part of Gargutron's Response for us to read. Now when the program runs: We send some sort of food to Gargutron and find out whether or not he can eat it.

Gargutron code step 4: Put the guesser in a loop

We can put the `input()` - `requests.get()` - `print()` code inside of a loop so that it will keep trying different ideas. Gargutron will start to give hints about what he considers to be good food; so by repeating tries you create a little game.

If you manage to write this code and win the Gargutron game (or if you get stuck): Let us know! (And use that effort to claim a Tier 1 prize if you like.)

Coding Basics

On this page are five programs for you to write. Use these to check your Python skills so far. If you get stuck: No worries! That's what the Python Bytes club and the coaches are for.

Coding Exercise 1: A simple mad-lib. Use the Python built-in function `input()` to ask for five pieces of information: A *name*, a *food item*, a *number*, a *toy*, and an *occupation*. Then print the mad-lib, filling in the blanks from the inputs: `One day _____ was finishing up a _____ when they had a great idea! They entered a nearby skyscraper, ran up the _____ stairs to the top and threw a _____ off the roof. It floated back down on a gust of wind and lightly bumped into a _____. No injuries were reported.`

Coding Exercise 2: How many characters? Define a string variable called `s` to have the value `"On day one I saw a bee."` Use the built-in function for length, called `len()`, to print the number of characters in this string. Your program should print out `"The string s contains _____ characters."`

Coding Exercise 3: Divisible by 17. Write a program that counts from 1 to 300. For each number: Check to see if each number is divisible by 17. If it is divisible by 17: Print the message `"_____ is divisible by 17."` You may find it helpful to use the remainder operator. In python this is the percent sign `%`. You can see how this remainder operator works by running the Python code `print(9 % 3, 9 % 2, 9 % 5)`.

Coding Exercise 4: Fibonacci. Write a program that calculates and prints the first 50 numbers of the Fibonacci sequence. The first seven of them are: 0, 1, 1, 2, 3, 5, 8.

Coding Exercise 5: Random falling. Use the import line `from random import randint` to make the 'random integer' function available to your program. Create a string which is a random number of spaces followed by the letter X. You can do this by simply multiplying the space character `" "` by an integer like `7` and then adding `+ "X"`.

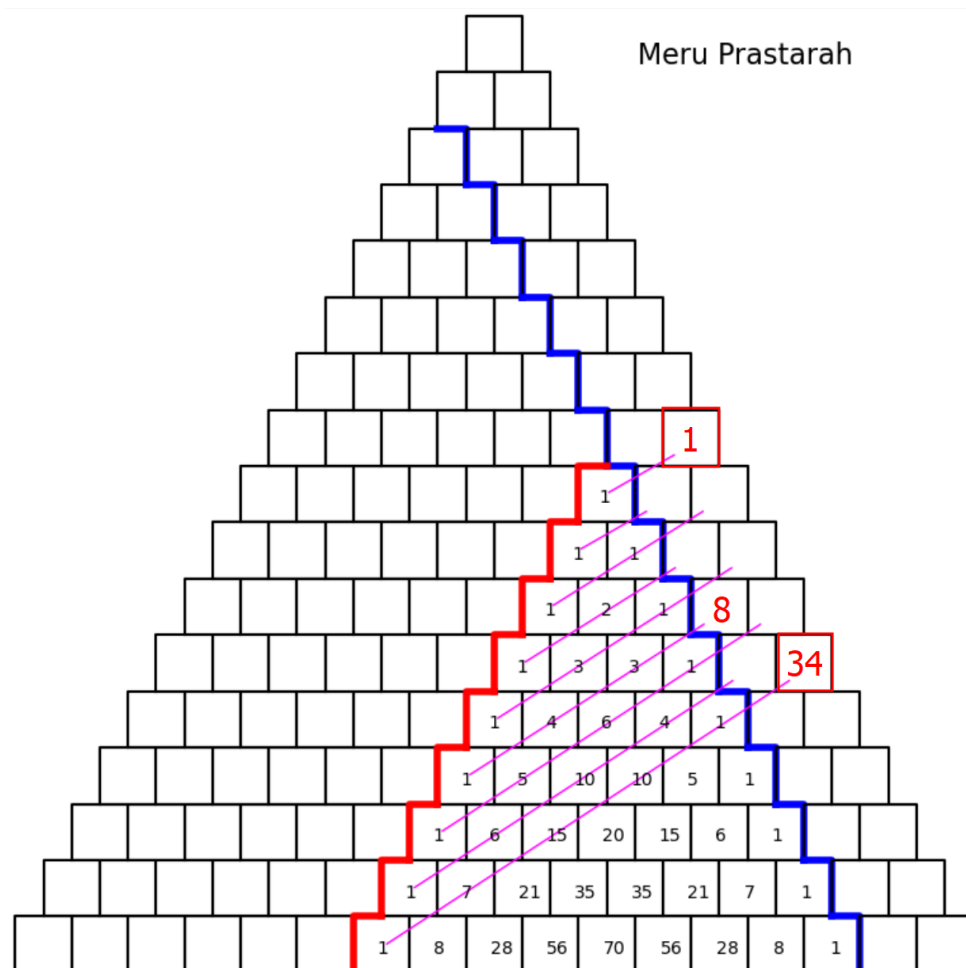
Remember: These exercises are a good check of your Python skills. If you are not sure how to do something: Ask a coach for a little help.

Meru Prastarah

Below is a drawing of Mount Meru, the mystic mathematical mountain at the center of the universe (so I am told). This mountain has other names as well (including “the Pascal triangle”) and fortunately for us the mountain has stairs built in so we can go up and down as we like.

Programming challenge: Create a program that prints the black numbers shown in the diagram below. Do not worry about printing the squares; just the numbers in triangle format will do. Start with the 1 at the top. Below that the next row will have two ones: 1 1.

Extra challenge: Three numbers are filled in on the right slopes of Mount Meru in red ink. They are 1, 8 and 34. Figure out how these numbers are connected to the numbers in black; and then fill in the missing numbers. Hint: The diagonal lines (magenta) are a *guide*.



Extra Extra Challenge: Can you figure out a way of continuing the red numbers all the way up to the top of the diagram?

Fibonacci ratios

Imagine a Fibonacci number, like say 5. This is followed by another number, namely 8. Let's make a fraction from them, called a Fibonacci *ratio*: It will be the next number (8) divided by the first number (5). The start number for Fibonacci is 0. The next number is 1. Add them to get the third number: 1 again. Now we have 0, 1, 1. Add 1 + 1 to get 2. Now we have 0, 1, 1, 2. Repeat! Now it's 0, 1, 1, 2, 3. Again! Now it's 0, 1, 1, 2, 3, 5... Let's calculate some Fibonacci ratios:

$$1/1 = 1$$

$$2/1 = 2$$

$$3/2 = 1.5$$

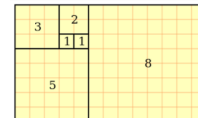
$$5/3 = 1.666666666666666666666666666666$$

$$8/5 = 1.6$$

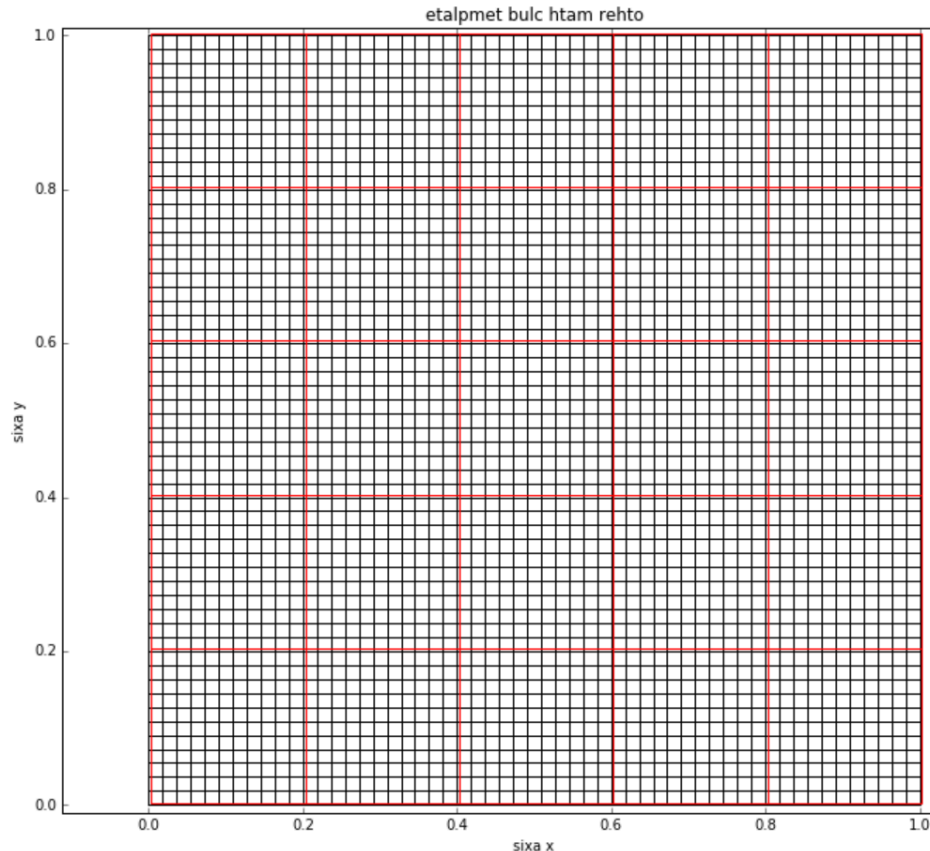
What happens to the ratio as you keep going on to bigger Fibonacci numbers?

Coding challenge: Write a program to print out the first 50 Fibonacci ratios.

Advanced coding challenge: Do the same starting with numbers *other than* 0 and 1. Compare the result with the Fibonacci ratio result.



Draw a Fibonacci spiral below. Here are the clues:



Graphing Grammar

This is a coding project. It will be challenging!

This Python code `s = "toothpaste"` means that the variable `s` contains a string of letters. This project begins with a very simple string. We will modify it to make it longer. Finally we read the result as *drawing instructions* for a turtle. For this we will need two rules. But first here is a warm-up exercise: Write a Python program that inputs a string and then prints every *other* letter of that string. Begin with this code:

```
s = input('Enter some text: ')
for c in s:
    print(c)
```

Onward: Let us begin with a simple string: `s = "x"`. `"x"` is both a word and a (short) sentence. To build longer sentences use **Rule S**: Go through the string one character at a time and build a new sentence. If the character in the starting string is `+` or `-` simply copy it into the new sentence. If the character is `x` or `y`: Replace it in the new sentence like so:

- `"x" -> "y+x+y"`
- `"y" -> "x-y-x"`

That is, every place you find `x` replace it with `y+x+y`. Doing this to `"x"` will produce a new sentence that is just `"y+x+y"`. Now apply Rule S again, to `"y+x+y"`. Now we end up with `"x-y-x+y+x+y-x-y-x"`. Our sentences are growing longer. Repeat Rule S three more times, each time making our result sentence longer. The end result will be quite long, almost 500 characters. Now to draw the end result using a turtle! How to do this? Answer: **Rule D**: Step through the string one character at a time. This time ignore the letters `x` and `y`; but for characters `+` and `-`:

- `+` means turn the turtle left 60 degrees then move forward 20 steps
- `-` means turn the turtle right 60 degrees then move forward 20 steps

Here is some code you may find useful.

```
t = Turtle()
for c in s:
    if c == '+': t.left(60)
    elif c == '-': t.right(60)
t.forward(20)
```

Once this is working feel free to experiment. You can change the forward amount, the angles, and you can even change Rule S and Rule D.

Logic and time

Make your best guess: What does this code do? What will it print? Then run it to see.

```
from time import time, sleep

a = 1
b = 1

print("First logic check")
print(a == b)
if a == b: print(True)
else:      print(False)

a = 0
print("Second logic check")
print(a == b)
print(not (a == b))
print(not ((a == b) or not (a == b)))

# Now let's work with time

toc = time()
print(toc)
sleep(3)
tic = time()
print(tic)
print('elapsed time (seconds) =', tic - toc)
print(tic - toc > 0)

toc = time()
for i in range(1000000):
    j = i
tic = time()
million_time = tic - toc
print('It took me', str(million_time), 'seconds to count to one million')

seconds_per_minute = 60
minutes_per_hour = 60
hours_per_day = 24
days_per_month = 30
months_per_year = 12

# Fix this next line of code

seconds_per_year = 1

print("Number of seconds per year is", seconds_per_year)
```

Turtles in pursuit

This code features a turtle chasing another turtle. This is *pursuit*. There is a simple version of a pursuit program and then a fancier version. Get a program to run and sort out how it works. Then try to write your own program to solve the Four Bugs problem described on the next page.

```
# simple version of turtle pursuit
t = Turtle()
p = Turtle()
t.up()
t.goto(-300, -150)
t.down()
for c in range(400):
    t.forward(2)
    d = p.towards(t)
    p.setheading(d)
    p.forward(2)
```

```
from turtle import Turtle    # more complicated version
from math import cos

def r_speed(s):              # experiment: go faster when you get closer
    # if s < 35: return 4
    # if s < 78: return 3
    return 2

q = Turtle()                 # this turtle follows a winding path
r = Turtle()                 # this turtle chases after the first turtle
q.pencolor("blue")
r.pencolor("red")

q.up(); q.goto(-300, -150); q.down()
r.up(); r.goto(-100, 350); r.down()

for t in range(400):
    q.setheading(20 + 40*cos(.015*t))
    q.forward(2)
    r.setheading(r.towards(q))
    r.forward(r_speed(r.distance(q)))
```

Turtle Pursuit Challenge

Write a pursuit program to solve the Four Bugs Problem: Four bugs named A, B, C, and D begin at the 4 corners of a square table, side length 1 meter. Each bug faces the bug at the next corner going clockwise; for example D is facing towards A. At noon the bugs begin walking: Each bug walks towards the bug it is facing. They take identical tiny bug steps and they all move at the same speed. As they go they adjust the *direction* of their walking progress (Why?) Do the bugs eventually meet? If so: Where? And supposing they meet: How far has each of them walked?

This Space Reserved For Next Week's Puzzle-Challenge (probably the Chaos Game)

Farmer Diophantus

There is a farmer named Diophantus who is very superstitious and who only likes integers. He goes to the market to buy some animals for his farm. He has \$100 in his pocket. Cows cost \$10 each. Pigs cost \$3 each. Chickens cost \$0.50 each. Because he is superstitious he must purchase exactly 100 animals with his 100 dollars... and he must have at least one cow, one pig, and one chicken. How many of each type of animal must he purchase?

Farmer Diophantus is superstitious as mentioned. Therefore he builds pens for his animal pens that are always right triangles with integer-length sides. He needs to build two pens and to save on fence material these two pens will share one side. Once he finishes the job (building two triangular pens that share one side) he finds he has built a total of 37 meters of fence. What are the dimensions of the two pens?

Watering horses

Farmer Diophantus owns a number of horses; and he takes good care of them. They mostly live in a large corral, so large that he can't see it all from his house. On the far side he can see some trees and he thinks there is a stream running through it somewhere. Anyway he is not even certain how many horses there are living in this corral.

Here is a problem about getting water for some horses. In the corner of the horse corral near to his house there is a water trough. Each morning Farmer Diophantus finds some number of horses standing by this trough (which has a water pump) waiting for water because they are thirsty. Horses drink exactly one gallon each morning. If the number of horses is h then Farmer Diophantus must pour exactly h gallons of water into the trough, no more and no less. (Remember he is superstitious.)

Suppose there are four horses at the watering trough one morning ($h=4$). Farmer Diophantus has two water buckets on hand, and both are unmarked. The first contains three gallons when full, the second contains eight gallons when full. Because they are unmarked they can not be partially filled with any accuracy. He can't get four gallons, for example, by filling the eight gallon bucket half way. The buckets can only be completely filled, to three gallons or to eight gallons.

How does Farmer Diophantus pour exactly four gallons of water into the trough?

(To be sure: Farmer Diophantus is ok with pouring water out onto the ground as necessary.)

(There are not one but two solutions to this problem.)

Python challenge: Write a Python program that solves the horse watering problem: The program begins with the number of horses h and the size in gallons of the two containers a and b . The program gives a recipe for how to get to h gallons of water.

Number Cards

There are three cards on a table. Each has a unique positive integer written on it but the number side of the card is face down. Let us call the three numbers a , b and c . We are informed that that a is less than b and b is less than c and $a+b+c=13$.

Now three girls (who know the conditions above) peek at the cards and make statements in the following order:

Anne says, after looking at a : I know a but I do not know b or c .

Caprice says, after looking at c : I know c but I do not know a or b .

Betsy says, after looking at b : I know b but I do not know a or c .

What is b ?